

改进后的网页购物系统软件过程定义文档

一、文档概述

1.1 编写背景

随着生成式人工智能（Generative AI）、大语言模型（Large Language Model, LLM）以及智能软件工程工具的发展，软件工程正在由传统的流程驱动模式向智能化、人机协同模式转变。

原《网页购物系统软件过程定义文档》参考ISO/IEC 12207、ISO/IEC 15504等国际标准，并结合四人项目团队、8~12周开发周期的实际情况，采用简化的瀑布+迭代混合开发模式完成软件开发活动。

在前期完成《AI技术应用场景匹配报告》和《AI技术应用验证报告》的基础上，本次对原有软件过程进行升级与重构，引入AI技术参与需求分析、系统设计、编码实现和测试验证等关键阶段，形成适用于小型团队的软件智能化开发过程。

1.2 过程目标

本过程旨在实现以下目标：

- 提高需求分析效率；
 - 提升系统设计质量；
 - 提高软件开发效率；
 - 提高测试覆盖率；
 - 降低重复性工作量；
 - 建立人机协同开发模式；
 - 在有限资源条件下提升软件质量。
-

1.3 过程原则

适用性原则

适用于4人项目团队。

轻量化原则

避免复杂流程和过度文档化。

AI辅助原则

AI承担重复性工作，人类负责决策和审核。

可验证原则

所有AI输出均需人工确认。

二、总体软件过程模型

2.1 生命周期模型

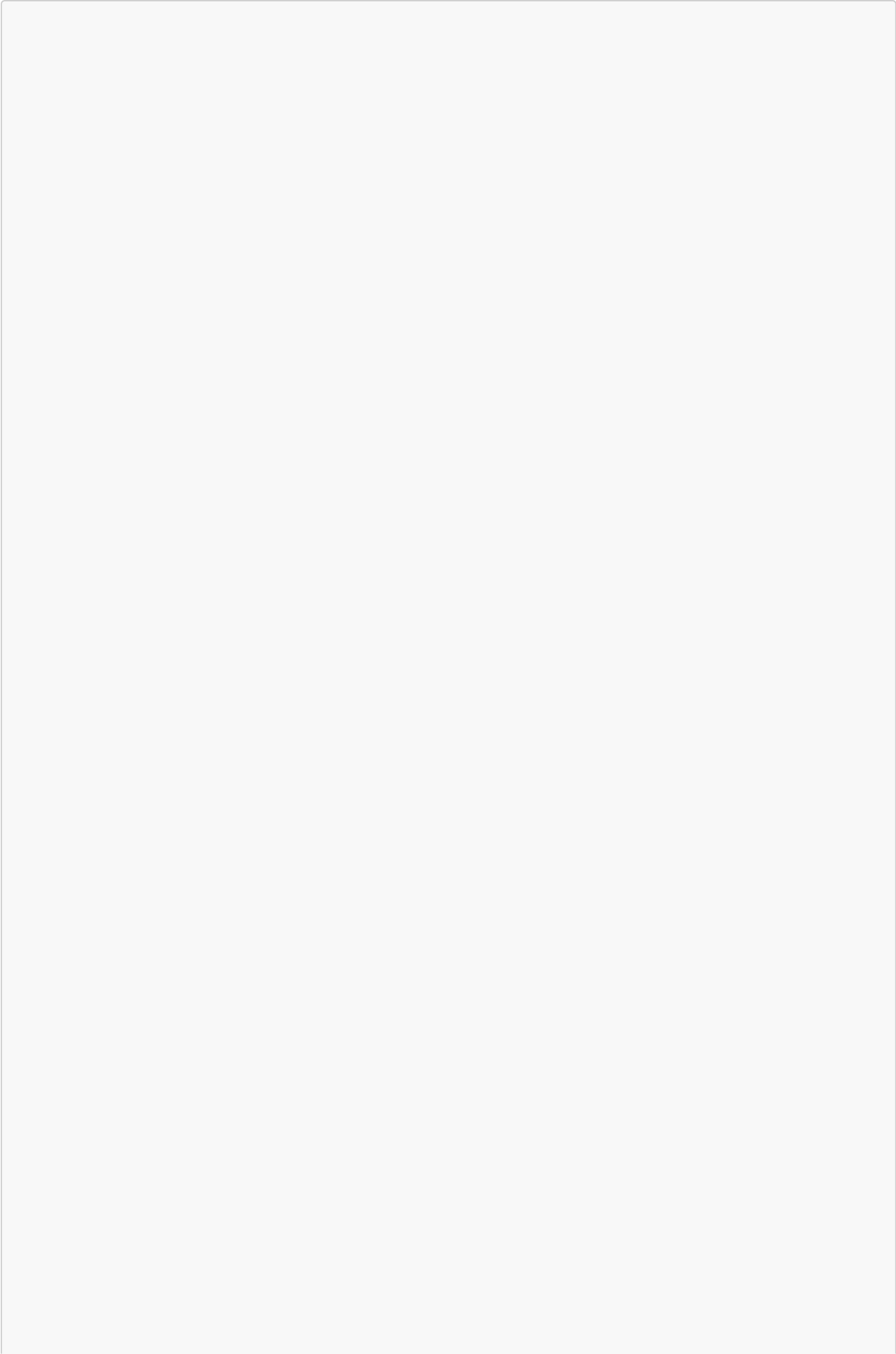
本项目采用：

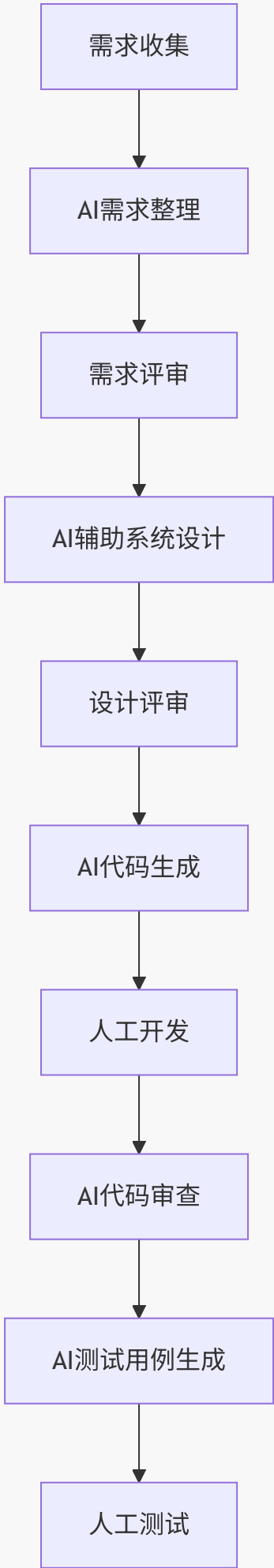
AI增强型瀑布+迭代混合模型

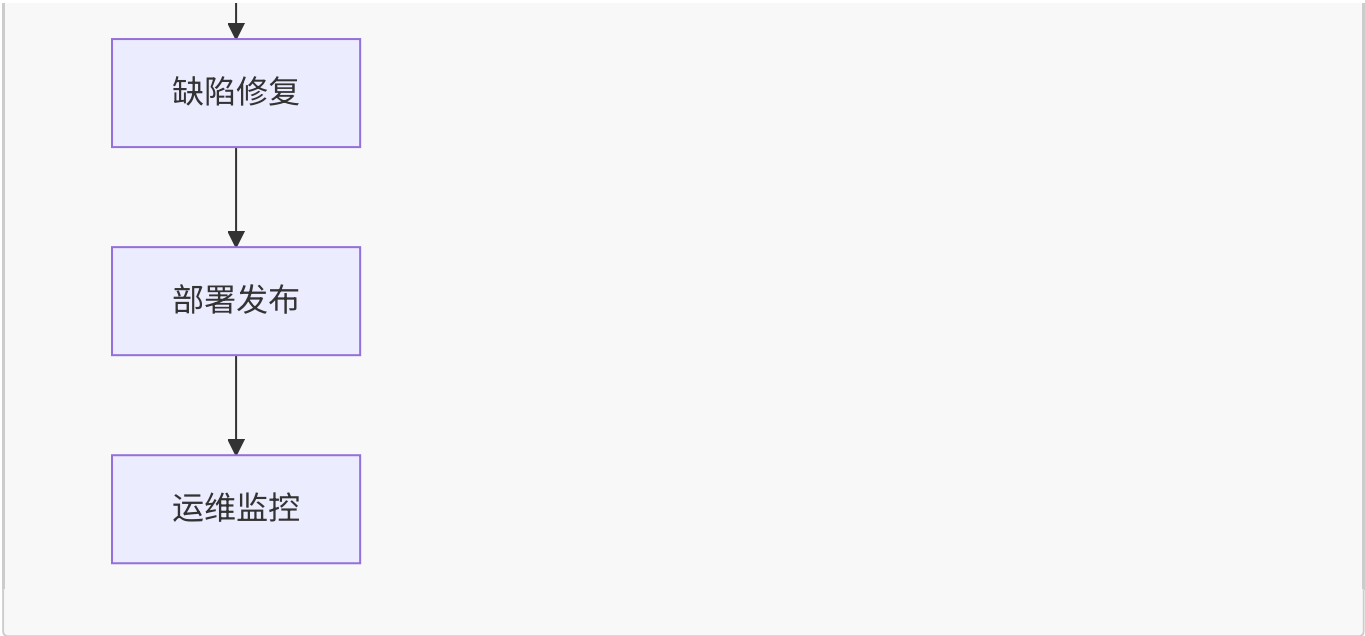
生命周期包括：

1. 需求获取与分析
2. 系统设计
3. 软件实现
4. 软件测试
5. 部署与维护

2.2 总体流程图







三、需求获取与分析过程（REQ-AI-01）

3.1 过程目标

明确系统功能需求与非功能需求，形成项目需求规格说明书（SRS）。

3.2 主要活动

活动1：需求收集

项目组讨论系统功能范围。

主要包括：

- 用户管理
- 商品管理
- 购物车
- 订单管理
- 支付功能

活动2：AI需求整理

利用ChatGPT整理需求。

输入内容：

- 功能列表
- 讨论记录
- 竞品分析结果

输出内容：

- 功能需求清单
- 非功能需求清单
- 用户故事

活动3：AI补充需求

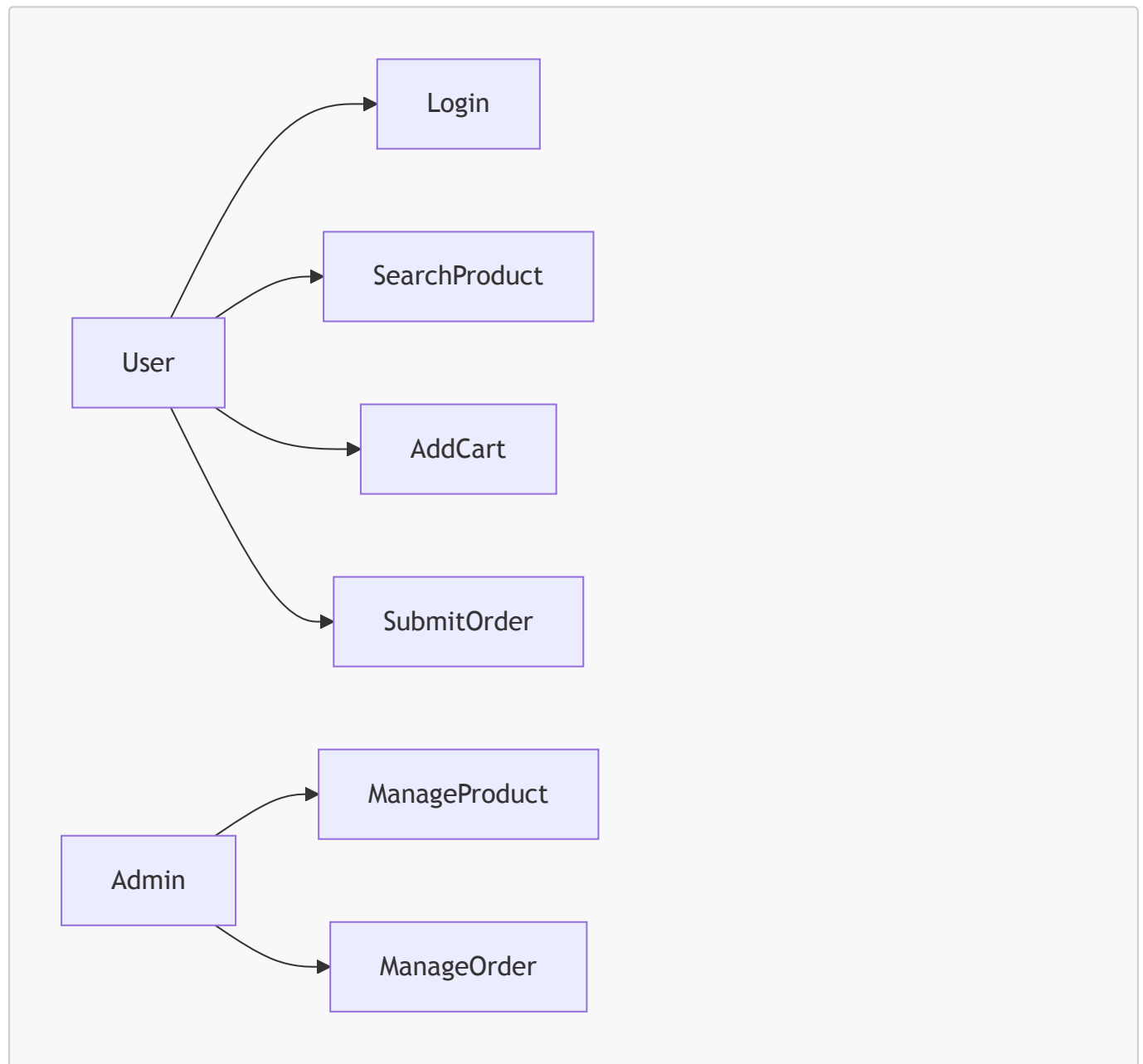
自动发现遗漏场景：

例如：

- 库存不足
- 重复支付
- 异常订单取消

活动4：AI生成用例图

示例：



活动5：人工评审

项目组确认需求正确性。

3.3 输出物

- SRS需求规格说明书
 - 用户故事集
 - 用例图
 - 功能需求清单
-

3.4 质量标准

- 需求完整率 $\geq 90\%$
 - 需求歧义率 $\leq 5\%$
 - 所有需求可追踪
-

四、系统设计过程（DES-AI-01）

4.1 过程目标

完成系统架构设计、数据库设计与接口设计。

4.2 主要活动

活动1：技术选型

AI根据项目特点推荐技术栈：

前端：

- Vue3

后端：

- Spring Boot

数据库：

- MySQL
-

活动2：数据库设计

AI根据需求自动生成ER模型。

核心实体：

- 用户
- 商品
- 订单
- 购物车
- 支付记录

活动3：接口设计

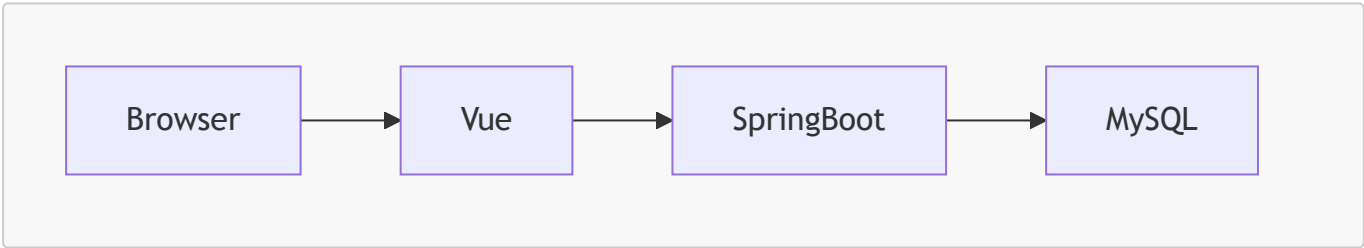
AI自动生成RESTful API文档。

示例：

```
POST /api/login
GET /api/product/list
POST /api/order/create
```

活动4：系统架构设计

生成系统架构图。



活动5：设计评审

开发组评审设计方案。

4.3 输出物

- 系统设计文档
- ER图
- API文档
- 架构图

4.4 质量标准

- 数据库满足第三范式
- API规范统一
- 架构满足扩展需求

五、软件实现过程（IMP-AI-01）

5.1 过程目标

完成系统开发与功能实现。

5.2 主要活动

活动1：环境搭建

建立：

- GitHub仓库
 - 开发环境
 - 数据库环境
-

活动2：AI代码生成

使用：

- GitHub Copilot
- ChatGPT

生成：

- CRUD代码
 - Controller层代码
 - Service层代码
-

活动3：人工开发

开发人员完成：

- 页面逻辑
 - 业务逻辑
 - 特殊功能实现
-

活动4：AI代码补全

实时生成：

- SQL语句
 - Java代码
 - Vue组件
-

活动5：AI代码审查

自动检查：

- 潜在Bug
 - 空指针异常
 - 安全问题
 - 性能问题
-

活动6：单元测试生成

AI生成JUnit测试代码。

5.3 输出物

- 项目源代码
 - Git提交记录
 - 单元测试代码
 - AI审查报告
-

5.4 质量标准

- 编译通过率100%
 - 单元测试覆盖率 $\geq 85\%$
 - 无严重安全漏洞
-

六、软件测试过程（TST-AI-01）

6.1 过程目标

验证系统满足需求。

发现并修复缺陷。

6.2 主要活动

活动1：AI测试用例生成

利用ChatGPT自动生成：

- 功能测试
 - 边界测试
 - 异常测试
 - 安全测试
-

活动2：测试执行

执行：

- 功能测试
 - 集成测试
 - 回归测试
-

活动3：AI缺陷分析

分析：

- 错误日志
 - 测试失败原因
 - 缺陷类型
-

活动4：缺陷修复

开发人员完成修复。

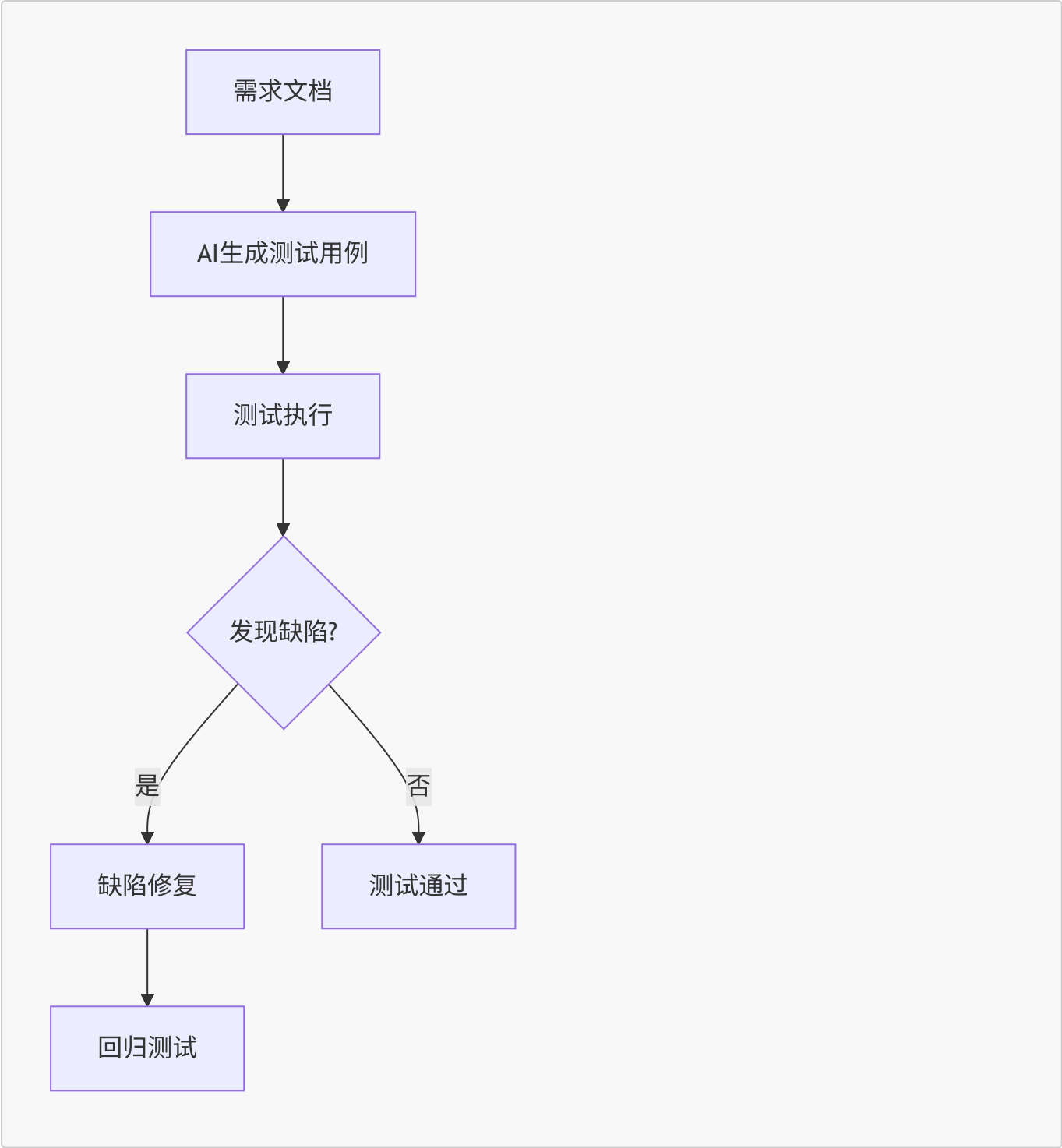
活动5：回归测试

验证问题已解决。

6.3 输出物

- 测试用例表
 - 测试报告
 - 缺陷记录
 - 回归测试报告
-

6.4 测试流程图



6.5 质量标准

- 测试覆盖率≥90%
- 缺陷修复率≥95%
- 严重缺陷数=0

七、部署与维护过程（DPL-AI-01）

7.1 过程目标

完成系统部署与交付。

7.2 主要活动

活动1：部署准备

生成：

- Docker配置
- 部署文档

活动2：部署实施

部署：

- Spring Boot服务
- MySQL数据库

活动3：AI辅助运维

监控：

- CPU利用率
- 内存占用率
- 接口响应时间

活动4：问题分析

AI分析异常日志。

定位故障原因。

7.3 输出物

- 部署文档
- 运维日志
- 性能分析报告

八、角色定义

角色	职责
项目经理	项目规划与管理
前端开发人员	Vue开发

角色	职责
后端开发人员	Spring Boot开发
测试负责人	测试组织与执行
AI过程协调员	管理AI工具使用
AI测试分析师	AI测试与缺陷分析

九、支持过程

配置管理

工具：

- Git
- GitHub

规范：

- 分支开发
- Pull Request评审

项目管理

工具：

- Notion
- GitHub Projects

活动：

- 周例会
- 进度跟踪

质量保证

包括：

- AI代码审查
- 文档互审
- 测试覆盖检查

十、交付物清单

阶段	交付物
需求分析	SRS、用例图
系统设计	ER图、架构图、API文档
软件实现	源代码、单元测试
软件测试	测试报告、缺陷记录
部署维护	部署文档、运维报告
AI活动	AI需求报告、AI测试报告、AI审查报告

十一、过程度量指标

11.1 原有指标

指标	目标值
需求完整率	≥90%
任务完成率	≥85%
缺陷修复率	≥95%
测试覆盖率	≥90%
系统可用性	正常运行

11.2 AI贡献度指标

新增以下指标：

AI生成代码占比目标：≥30%

AI生成测试用例占比目标：≥70%

AI节约工时目标：≥40%

AI输出采纳率目标：≥60%

AI辅助缺陷发现率目标：≥20%

十二、风险控制

数据安全风险措施：

- 不上传敏感数据
- 数据脱敏处理

模型幻觉风险措施：

- 人工审核
- 多轮验证

技术依赖风险措施：

- 保留人工决策
- 强化团队学习

十三、总结

本软件过程在原有简化瀑布+迭代模型基础上，引入了ChatGPT、GitHub Copilot等AI工具，实现了需求分析、系统设计、编码实现、测试验证以及运维监控等多个阶段的智能化升级。

改进后的过程充分体现"AI辅助、人类主导"的思想，在保证软件工程规范性的同时，大幅提高了开发效率和软件质量。对于项目而言，该过程具有较强的可执行性和实践价值，可作为智能化软件开发过程的参考范例。